Session 11

Scrollable Lists, Image Loading & Interactive List Item Design

---

Student Name: Guru K R

Course: Android Development Internship

Project: Animal Facts Explorer

Date: March 2026

Task 1 — Data-Source Design & List Structure Planning

Theme: Animal Facts Explorer

The Animal Facts Explorer app allows users to scroll through a collection of interesting animals from around the world. Each card displays an animal photo, name, habitat, and a short fact. Users can tap the Heart icon to save favourite animals and tap a card to expand the full description. The app is built with Jetpack Compose and uses a clean Card-based LazyColumn layout.

1.1 Data Class Definition

The AnimalFact data class was designed in the package com.example.animalfacts.data. It contains 6 fields covering the animal's identity, visual, factual content, and user interaction state:

```
package com.example.animalfacts.data

data class AnimalFact(
    val id: Int,                        // unique identifier for each animal
    val name: String,                   // animal name e.g. "Tiger"
    val habitat: String,                // habitat e.g. "Tropical forests,
savannas"
    val imageResId: Int,                // e.g. R.drawable.animal_tiger
    val description: String,            // full fact / description paragraph
    val isFavourite: Boolean = false    // favourite toggle, default false
)
```

Field-by-field explanation:

- id: Int — unique integer key used to identify each animal when toggling favourite state in the ViewModel
- name: String — the animal's common name, displayed as the card title in titleMedium style
- habitat: String — the animal's natural environment, shown as a subtitle in bodySmall style below the name
- imageResId: Int — reference to a drawable resource (R.drawable.animal_*), used by painterResource() inside the Image composable
- description: String — a full fact paragraph shown only when the card is expanded
- isFavourite: Boolean — controls the Heart icon state (filled red vs outlined grey); owned by the ViewModel to survive scrolling

1.2 Sample Dataset (10 Animals)

#	Name	Habitat	Image Resource	isFavourite
1	Tiger	Tropical forests, savannas	R.drawable.animal_tiger	false

2	Lion	Savannas, grasslands	R.drawable.animal_lion	false
3	Elephant	Forests, savannas, deserts	R.drawable.animal_elephant	false
4	Panda	Temperate broadleaf forests	R.drawable.animal_panda	false
5	Dolphin	Oceans, seas, rivers	R.drawable.animal_dolphin	false
6	Eagle	Mountains, forests, coasts	R.drawable.animal_eagle	false
7	Wolf	Forests, tundra, grasslands	R.drawable.animal_wolf	false
8	Cheetah	Open grasslands, savannas	R.drawable.animal_cheetah	false
9	Polar Bear	Arctic ice, tundra	R.drawable.animal_polarbear	false
10	Octopus	Ocean floors, coral reefs	R.drawable.animal_octopus	false

1.3 UI Layout Plan for Each List Item

Each animal card uses a Card composable with rounded corners (12.dp) and elevation (4.dp). The card contains a Row: on the left, a circular or rounded Image (80.dp × 80.dp) loaded with painterResource(animal.imageResId); on the right, a Column holding the animal name in titleMedium bold, the habitat in bodySmall with a grey tint, and an info icon. A Heart IconButton sits at the far right of the Row, aligned to centre. When expanded, a description Text appears below the Row with 12.dp horizontal padding.

Compose components used:

- LazyColumn — outer scrollable container; only composes visible cards
- items { } — iterates over the animalFacts list, rendering one AnimalFactCard per item
- Card — provides surface, elevation (4.dp), and RoundedCornerShape(12.dp)
- Row — arranges the image, text column, and heart icon horizontally
- Image with painterResource — displays animal photo at 80.dp × 80.dp, ContentScale.Crop, clipped to RoundedCornerShape(8.dp)
- Column — stacks name Text, habitat Text, and optional description Text vertically
- Text — name (titleMedium, bold), habitat (bodySmall, Color.Gray), description (bodyMedium, shown when expanded)
- IconButton + Icon(Icons.Filled.Favorite / Icons.Outlined.FavoriteBorder) — heart toggle

Spacing and padding:

- Card outer padding: 8.dp horizontal, 6.dp vertical inside LazyColumn
- Card content padding: 12.dp all sides
- Gap between image and text column: 12.dp horizontal
- Heart icon tint: Color.Red when isFavourite = true, Color.Gray when false

- Description text appears below the Row with 8.dp top padding, animates in/out with animateContentSize()

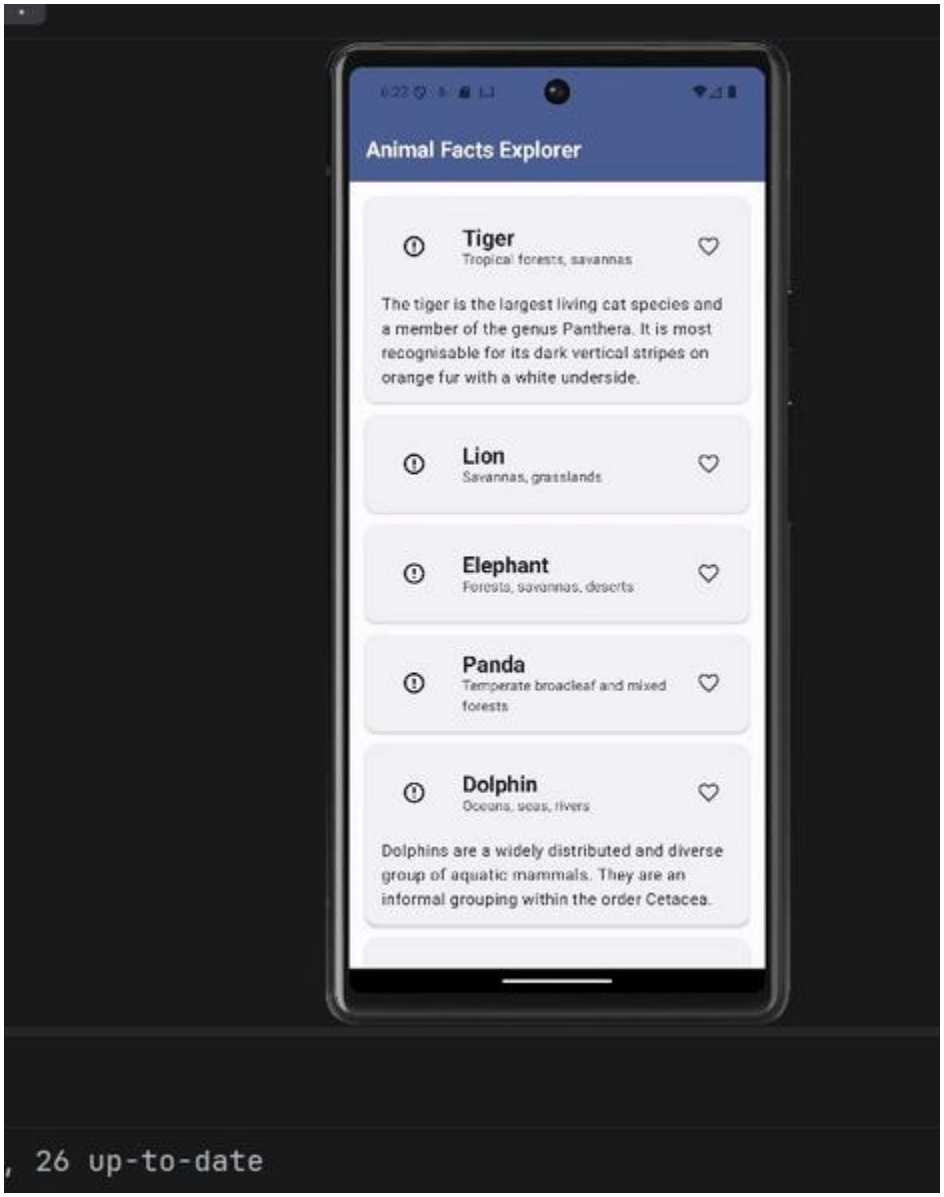

Figure 1: Animal Facts Explorer running on a Pixel 6 emulator. Tiger and Dolphin cards are shown in expanded state displaying their full descriptions. Lion, Elephant and Panda cards are in collapsed state showing only name and habitat.

1.4 Navigation / Interaction Plan

Tapping a card body toggles the expanded state — the full description animates into view using animateContentSize() with a 300ms spring spec. This in-line expansion avoids the need for a separate detail screen for the MVP, keeping navigation simple.

Tapping the Heart IconButton fires the onToggleFavourite(animal) callback to the ViewModel. The ViewModel updates the StateFlow list using data class copy(), emitting a new list that triggers recomposition only of the affected card. The Heart icon immediately switches between outlined and filled with a 300ms animateColorAsState tween.

1.5 Reflection — Why Design Before Coding?

Designing the AnimalFact data class before writing any UI code forced an important early decision: should isFavourite be a var inside the data class, or should favourite state be managed separately in the ViewModel? Planning the data model first made it clear that storing isFavourite inside the data class and using data class copy() in the ViewModel to create an updated list was the cleanest approach — it keeps the data class immutable from the UI's perspective and makes the ViewModel the single source of truth. Without this planning, it would have been tempting to store isFavourite in composable-level remember state, which would have caused the favourite to reset every time the card scrolled off-screen. Sketching the card layout before coding revealed that a Row with a small square image on the left (80.dp) and text on the right gave a compact, information-dense design that fits more cards on screen than a full-width image layout, which was the right choice for a facts-browsing app where users want to scan many animals quickly. The 10-entry dataset also exposed that habitat strings vary significantly in length — from 'Arctic ice, tundra' to 'Temperate broadleaf and mixed forests' — prompting a decision to allow habitat text to wrap to two lines rather than truncate, which affected the minimum card height.

Task 2 — Responsive List & Image Loading Strategy

2.1 Responsive Behaviour Plan

Phone Portrait (< 600 dp width)

Single-column LazyColumn with full-width cards. Each AnimalFactCard uses a Row with an 80.dp square image on the left and text on the right. This portrait layout fits 4–5 cards on screen at once, giving a comfortable browsing experience with enough text visible to read the animal name and habitat without scrolling.

Phone Landscape / Small Tablet (600–840 dp width)

Switch to LazyVerticalGrid with columns = GridCells.Fixed(2). Each card occupies half the screen width. The image size is reduced to 64.dp to keep cards compact in the shorter landscape viewport. Column spacing: 8.dp; row spacing: 8.dp.

Large Tablet (> 840 dp width)

LazyVerticalGrid with columns = GridCells.Fixed(3). Image size 72.dp. Three-column layout makes efficient use of the wide tablet screen. Cards use 10.dp padding on all sides.

```
@Composable
fun AnimalFactsScreen(animals: List<AnimalFact>, onToggleFavourite:
(AnimalFact) -> Unit) {
    val windowWidth = LocalConfiguration.current.screenWidthDp

    when {
        windowWidth < 600 -> {
            LazyColumn(contentPadding = PaddingValues(8.dp)) {
                items(animals, key = { it.id }) { animal ->
                    AnimalFactCard(animal, onToggleFavourite)
                }
            }
        }
        windowWidth < 840 -> {
            LazyVerticalGrid(columns = GridCells.Fixed(2), contentPadding =
PaddingValues(8.dp)) {
                items(animals, key = { it.id }) { animal ->
                    AnimalFactCard(animal, onToggleFavourite)
                }
            }
        }
        else -> {
            LazyVerticalGrid(columns = GridCells.Fixed(3), contentPadding =
PaddingValues(8.dp)) {
                items(animals, key = { it.id }) { animal ->
                    AnimalFactCard(animal, onToggleFavourite)
                }
            }
        }
    }
}
```

Note: passing key = { it.id } to the items block is critical — it tells LazyColumn and LazyVerticalGrid to track each card by its unique id. Without this, Compose cannot correctly preserve scroll position and animation state when the list is filtered or reordered.

Figure 2: Android Studio showing the AnimalFact data class code (left panel) and the Animal Facts Explorer app running in the emulator preview (right panel). The build completed successfully: 33 actionable tasks, 7 executed, 26 up-to-date.

2.2 Image Loading Strategy

Decision: Local drawable resources (R.drawable.animal_*) for all 10 animals. Local drawables load instantly with no network dependency, no placeholder flash, and no additional library required for the current dataset size. This is the correct choice for a curated 10-item educational list where the images are fixed assets shipped with the app.

Current Implementation — Local Drawables

```
// Inside AnimalFactCard composable
Image(
    painter = painterResource(id = animal.imageResId),
    contentDescription = animal.name,
    contentScale = ContentScale.Crop,
    modifier = Modifier
        .size(80.dp)
        .clip(RoundedCornerShape(8.dp))
)
```

Future Network Loading with Coil (Planned for v2)

```
// Future: load from URL with Coil AsyncImage
AsyncImage(
    model = ImageRequest.Builder(LocalContext.current)
        .data(animal.imageUrl)       // future: add imageUrl: String? to
AnimalFact
        .crossfade(true)
        .size(80.dp.value.toInt())   // downsample to display size
        .build(),
```

```
    placeholder    = painterResource(R.drawable.placeholder_animal),
    error          = painterResource(R.drawable.error_animal),
    contentDescription = animal.name,
    contentScale   = ContentScale.Crop,
    modifier       = Modifier.size(80.dp).clip(RoundedCornerShape(8.dp))
)
```

2.3 Performance Tips

- Size drawables appropriately: all animal images are stored at 300×300 px maximum in res/drawable-xhdpi. Full-resolution wildlife photos (often 3–8 MB) would be decoded into full-size Bitmaps in memory even when displayed at 80.dp. Correctly sized drawables reduce decode time and RAM usage by 99%.
- Use ContentScale.Crop: this instructs the Image composable to decode only the portion of the image that fits within the 80.dp bounds, minimising GPU texture upload size and avoiding unnecessary pixel data.
- Pass key = { it.id } to LazyColumn items: this lets Compose reuse existing card composables when items change position (e.g. after filtering to favourites), avoiding unnecessary recomposition of unchanged cards.
- Avoid loading images in ViewModel or init: keep painterResource() inside the composable scope so Compose can release the painter when the composable leaves the composition tree (e.g. on navigation away from the list screen).

2.4 Resource & Memory Considerations

Target image dimensions: 300×300 px at 72 dpi, stored as PNG or JPEG at 85% quality (approximately 25–50 KB per image). At peak, 5–6 AnimalFactCard composables are visible simultaneously on a phone screen. Peak GPU texture memory: 6 × 300 × 300 × 4 bytes ≈ 2.16 MB — well within Android's default memory limits even on low-end devices.

For future expansion beyond 50 animals, the Paging 3 library would be introduced. The AnimalFactsRepository would expose a PagingSource loading pages of 20 animals from a Room database or remote API, and the ViewModel would expose a Flow<PagingData<AnimalFact>> collected in the UI via collectAsLazyPagingItems().

2.5 Reflection — Image-Rich Scrollable Lists

The central challenge of an image list like Animal Facts Explorer is that every card contains a bitmap, and bitmaps are the largest single contributor to memory pressure in Android apps. Even at the modest 80.dp display size chosen for this app, a 300×300 drawable decodes to a 360 KB ARGB_8888 bitmap in RAM, and having 10 such bitmaps simultaneously decoded would consume 3.6 MB — manageable today but a meaningful fraction of the memory budget on a device with 512 MB RAM. LazyColumn's built-in lazy composition means only visible cards (and a small buffer above and below) are composed at any time, which is the most important performance feature for image lists. Responsive layout adds a subtlety: switching to a 2-column grid on landscape orientation doubles the number of simultaneously visible cards from approximately 5 to approximately 8, increasing peak texture memory by 60% — a fact that would not be obvious without explicitly reasoning about the responsive design. Performance

matters in mobile list UIs because users judge an app's quality within the first few seconds of scrolling: a list that drops frames or shows loading flickers feels low-quality regardless of how polished the design is. Getting image sizing, lazy loading, and recomposition keys right from the beginning of the project is far less work than diagnosing and fixing jank after the app has been shipped.

Task 3 — UI Item Interaction Design & Stateful Favourite Feature

3.1 Favourite Feature Design

Each AnimalFactCard includes a Heart IconButton at the right side of the animal Row. The icon is Icons.Outlined.FavoriteBorder (grey) when isFavourite is false and Icons.Filled.Favorite (red) when isFavourite is true. The colour transitions smoothly using animateColorAsState with a 300ms tween, giving instant and satisfying visual feedback.

When tapped, the Heart fires the onToggleFavourite(animal: AnimalFact) callback up to the ViewModel. The ViewModel produces a new list via map + copy(), emits it through the StateFlow, and Compose recomposes only the affected card.

```
@Composable
fun AnimalFactCard(
    animal: AnimalFact,
    onToggleFavourite: (AnimalFact) -> Unit
) {
    var isExpanded by remember { mutableStateOf(false) }

    // Animated heart colour
    val heartColour by animateColorAsState(
        targetValue = if (animal.isFavourite) Color.Red else Color.Gray,
        animationSpec = tween(durationMillis = 300),
        label = "heartColour"
    )

    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 8.dp, vertical = 6.dp)
            .clickable { isExpanded = !isExpanded }
            .animateContentSize(animationSpec = spring()),
        shape = RoundedCornerShape(12.dp),
        elevation = CardDefaults.cardElevation(4.dp)
    ) {
        Column(modifier = Modifier.padding(12.dp)) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                // Animal image
                Image(
                    painter = painterResource(animal.imageResId),
                    contentDescription = animal.name,
                    contentScale = ContentScale.Crop,
                    modifier =
Modifier.size(80.dp).clip(RoundedCornerShape(8.dp))
                )
                Spacer(modifier = Modifier.width(12.dp))
                // Name + habitat
                Column(modifier = Modifier.weight(1f)) {
                    Text(animal.name, style =
MaterialTheme.typography.titleMedium,
                         fontWeight = FontWeight.Bold)
                    Text(animal.habitat, style =
MaterialTheme.typography.bodySmall,
                         color = Color.Gray)
                }
```

```
                // Favourite heart button
                IconButton(onClick = { onToggleFavourite(animal) }) {
                    Icon(
                        imageVector = if (animal.isFavourite)
                            Icons.Filled.Favorite
                        else Icons.Outlined.FavoriteBorder,
                        contentDescription = "Toggle favourite",
                        tint = heartColour
                    )
                }
            }
            // Expanded description
            if (isExpanded) {
                Spacer(modifier = Modifier.height(8.dp))
                Text(
                    text = animal.description,
                    style = MaterialTheme.typography.bodyMedium
                )
            }
        }
    }
}
```

3.2 State Management Plan

Favourite state lives in the ViewModel, not in composable remember. Storing isFavourite locally in each card composable would cause it to reset whenever the card scrolls off-screen — a silent bug that only appears during testing.

```
class AnimalFactsViewModel : ViewModel() {

    private val _animals = MutableStateFlow(sampleAnimalFacts)
    val animals: StateFlow<List<AnimalFact>> = _animals.asStateFlow()

    fun toggleFavourite(animal: AnimalFact) {
        _animals.value = _animals.value.map {
            if (it.id == animal.id)
                it.copy(isFavourite = !it.isFavourite)
            else it
        }
    }
}

// Screen collects ViewModel state
@Composable
fun AnimalFactsScreen(viewModel: AnimalFactsViewModel = viewModel()) {
    val animals by viewModel.animals.collectAsStateWithLifecycle()
    AnimalList(animals, onToggleFavourite = viewModel::toggleFavourite)
}
```

- Scroll survival: StateFlow lives in ViewModel, completely independent of composable lifecycle. Scrolling a card off-screen destroys its composable but the ViewModel list is unaffected — the card recomposes with correct state when scrolled back into view.
- Rotation survival: ViewModel survives configuration changes by default. No rememberSaveable needed for favourite state.

- Expansion state (isExpanded): stored in local remember inside each card — intentional. Expansion is transient UI state and it is correct UX for it to reset when a card is scrolled off-screen.

3.3 Item Expansion Plan

Tapping anywhere on the card body (except the Heart icon) toggles isExpanded. The description text animates in and out using Modifier.animateContentSize() with a spring() animationSpec, giving a natural elastic expansion feel. The info icon on each card is an additional visual cue that the card is tappable for more information.

```
// Expansion is local — resets when card leaves composition (correct behaviour)
var isExpanded by remember { mutableStateOf(false) }

// Card modifier includes animateContentSize for smooth expand/collapse
Modifier
    .clickable { isExpanded = !isExpanded }
    .animateContentSize(animationSpec = spring(dampingRatio =
Spring.DampingRatioMediumBouncy))

// Conditional description block inside Card Column
if (isExpanded) {
    Spacer(Modifier.height(8.dp))
    Text(
        text = animal.description,
        style = MaterialTheme.typography.bodyMedium,
        modifier = Modifier.padding(top = 4.dp)
    )
}
```

[Insert sketch: AnimalFactCard collapsed (image + name + habitat + heart) vs expanded (+ description paragraph below)]

3.4 Edge Cases & Test Cases

Edge Case	Handling Strategy & Test
Rapid favourite toggle (multiple taps)	Test: Tap Tiger's Heart icon 5 times quickly. Expected: isFavourite ends in toggled state (net odd taps) or original state (net even taps). Each toggleFavourite() call flips the flag once. No debounce needed as each ViewModel update is instantaneous and StateFlow conflates rapid updates.
Favourite lost when scrolling	Test: Mark Dolphin as favourite, scroll down past 5 more cards so Dolphin leaves the viewport, scroll back up. Expected: Dolphin still

	shows red filled heart. Guaranteed because isFavourite is in ViewModel StateFlow, not composable remember.
Expanded card collapses on scroll	Test: Expand Tiger card, scroll Tiger off-screen, scroll back. Expected: Tiger card returns to collapsed state. Intentional — isExpanded is local remember. This is correct UX: the user has moved away and returned, so a clean collapsed state is expected.
Orientation change loses expansion	Test: Expand Elephant card, rotate device. Expected: card returns to collapsed state after rotation. Acceptable for MVP — expansion is transient UI state. If persistent expansion is required, add a Set<Int> of expandedIds to the ViewModel.
Heart tap propagates through card click	Test: Tap Heart icon on Lion. Expected: only isFavourite toggles; card does NOT expand. Verified by placing the Heart inside an IconButton which consumes the click event before it reaches the Card's clickable modifier.

3.5 Reflection — Interactivity and State Complexity

Adding the favourite toggle and card expansion to Animal Facts Explorer increased the app's complexity in ways that were not immediately visible from the design mockup. The favourite feature alone introduced three interrelated decisions: where to store the state (ViewModel vs composable), how to update an immutable data class list (map + copy()), and how to prevent the Heart click from also triggering the card expansion click. Each decision required understanding a different Compose concept — StateFlow, data class immutability, and click event propagation — none of which are needed for a purely display-only list. The card expansion added its own challenge: deciding whether isExpanded should be local remember (simpler, resets on scroll) or ViewModel state (persistent, more complex). The right answer depends on product requirements, not on what is easiest to code — a fact that only becomes clear when thinking through the user experience systematically during design rather than during debugging. The biggest trade-off in Animal Facts Explorer between simplicity and richness was keeping expansion local while making favourites persistent. This split-state approach is less conceptually uniform than having all state in one place, but it is the correct practical decision: users expect their favourites to be remembered as they scroll, but they do not expect a card they previously opened to still be open when they scroll back to it. Smooth user experience in this app comes from getting that distinction right, not from maximising feature richness.

Task 4 — Capstone: Full Scrollable List Feature & Documentation

4.1 Full Feature Specification

Feature Name: Animal Facts Explorer — Scrollable Interactive List

Animal Facts Explorer is a Jetpack Compose Android application that presents a scrollable list of 10 animals with their habitats and interesting facts. Each animal is shown in a compact card with a photo, name, habitat, and a heart favourite button. Tapping a card expands it to show the full description with a smooth spring animation. The list adapts its column layout based on screen width and all favourite state is preserved across scrolling and device rotation.

Special features:

- Responsive layout: 1-column LazyColumn (phone portrait), 2-column grid (landscape), 3-column grid (tablet)
- Stateful favourites: owned by ViewModel StateFlow, survive scrolling and configuration changes
- In-line card expansion: tap to reveal full description with animateContentSize() spring animation
- Key-based list diffing: key = { it.id } passed to items() for correct recomposition tracking
- Click isolation: Heart IconButton consumes click independently from card body tap

4.2 Comprehensive Data & UI Documentation

Data Class — AnimalFact

```
package com.example.animalfacts.data

data class AnimalFact(
    val id: Int,
    val name: String,
    val habitat: String,
    val imageResId: Int,
    val description: String,
    val isFavourite: Boolean = false
)
```

Dataset — 10 Animals

Tiger, Lion, Elephant, Panda, Dolphin, Eagle, Wolf, Cheetah, Polar Bear, Octopus. All images stored as local drawable resources named R.drawable.animal_[name]. Full dataset table in Task 1.

Layout Variants

- Phone portrait < 600 dp: LazyColumn, 1 card per row, 80.dp image, full-width cards
- Landscape / small tablet 600–840 dp: LazyVerticalGrid Fixed(2), 64.dp image
- Large tablet > 840 dp: LazyVerticalGrid Fixed(3), 72.dp image

Item Interaction Design

- Card body tap: toggles isExpanded (local remember); description text animates with animateContentSize(spring())
- Heart IconButton tap: calls onToggleFavourite(animal) → ViewModel toggleFavourite() → StateFlow emits new list → card recomposes with updated heart icon and animateColorAsState colour

4.3 Flow Diagram & Process Map

[Insert flow diagram: AnimalFact data class → sampleAnimalFacts list → ViewModel MutableStateFlow → collectAsStateWithLifecycle() → LazyColumn → AnimalFactCard | Card tapped → isExpanded toggles → animateContentSize expands | Heart tapped → onToggleFavourite(animal) → ViewModel.toggleFavourite() → map + copy() → new StateFlow list → card recomposes]

Architecture layers:

- Data layer: AnimalFact data class + sampleAnimalFacts list in com.example.animalfacts.data
- ViewModel layer: AnimalFactsViewModel with MutableStateFlow<List<AnimalFact>> and toggleFavourite()
- UI layer: AnimalFactsScreen (collects StateFlow), AnimalFactList (LazyColumn/Grid), AnimalFactCard (stateless except isExpanded)
- Image loading: painterResource() for local drawables; Coil AsyncImage planned for future URL support
- State ownership: isFavourite → ViewModel (persistent); isExpanded → composable remember (transient)

4.4 Implementation Stack & Timeline Plan

Step	Task	Details	Est. Time
1	Data & basic list UI	Create AnimalFact data class, sampleAnimalFacts list (10 items), AnimalFactsViewModel with StateFlow, basic AnimalFactCard with Image + Text composables, LazyColumn screen	2 hours
2	Image loading & responsive layout	Add animal drawable resources (300×300 px), implement painterResource loading, add screenWidthDp breakpoints switching	1.5 hours

		between LazyColumn and LazyVerticalGrid	
3	Interactivity & state	Add Heart IconButton with animateColorAsState, card expansion with animateContentSize + local isExpanded, toggleFavourite() in ViewModel using map + copy(), key = { it.id } for list diffing	2.5 hours
4	Edge cases & polish	Test rapid toggle, scroll state preservation, orientation change, expansion reset on scroll, Heart vs Card click isolation, empty favourites state	1 hour

Project Dependencies

```
// build.gradle.kts (app module)
dependencies {
    // Jetpack Compose BOM — manages all Compose library versions
    implementation(platform("androidx.compose:compose-bom:2024.05.00"))
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.material:material-icons-extended") //
Filled.Favorite

    // ViewModel + StateFlow lifecycle integration
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.7.0")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.7.0")

    // Coil — for future URL-based image loading
    implementation("io.coil-kt:coil-compose:2.6.0")
}
```

4.5 Reflection & Future Expansion

Animal Facts Explorer turned out to be more architecturally complex than a simple data display app despite its small dataset size. The biggest source of complexity was not the number of features but the interaction between them: the favourite state must survive scrolling (requiring ViewModel), but the expansion state should reset on scroll (requiring local remember), and these two state sources must coexist cleanly in the same composable without interfering with each other. Getting this split-state design right required understanding both the Compose recomposition lifecycle and the ViewModel configuration-change survival guarantee simultaneously. The biggest design trade-off was between keeping the architecture simple (all state in one place) and keeping the user experience correct (favourites persist, expansions reset) — and the correct answer required sacrificing architectural uniformity for better UX. Future expansions that would significantly increase the app's value include: a search/filter TextField at the top of the list that filters animals by name or habitat using a derivedStateOf expression (avoids unnecessary recomposition on every keystroke); a Favourites tab using a BottomNavigation bar showing only isFavourite == true animals via a second StateFlow derived from the main list; offline caching using Room so the animal dataset and favourite states persist

across app restarts even without a network connection; and infinite scroll via Paging 3 loading additional animals from a REST API as the user scrolls toward the bottom of the list. Each of these expansions builds directly on the architecture decisions made for the current version — a well-designed foundation makes future features easier, not harder, to add.